

POLITECNICO
MILANO 1863

Best Bike Paths – DD Document

DD document for the
Software Engineering 2 course assignment

Authors: **Alessandro Burato, Gabriele Crimella, Davide Faccini**

Teacher: Elisabetta Di Nitto
Academic Year: 2025-26

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms, Abbreviations . . . 1
1.4 Reference Documents . . . 2
1.5 Document Structure . . . 2

2 Architectural Design **3**
2.1 Overview . . . 3
2.1.1 Client Side . . . 3
2.1.2 Server Side . . . 3
2.2 Component View . . . 5
2.2.1 High Level Diagram . . . 5
2.2.2 Low Level Diagram . . . 6
2.3 Deployment View . . . 9
2.4 Runtime View . . . 10
2.4.1 User Login . . . 10
2.4.2 User Registration . . . 12
2.4.3 User Logout . . . 13
2.4.4 Manual Trip Recording . . . 15
2.4.5 Automated Trip Recording . . . 17
2.4.6 Personal Trip Storage . . . 19
2.4.7 Path Search . . . 21
2.4.8 Browsing of Published Paths . . . 23
2.4.9 Obstacles Detection . . . 24
2.5 Component Interfaces . . . 26
2.6 Styles and Patterns . . . 26
2.7 Other Design Decisions . . . 27

2.7.1 JWT 27
2.7.2 Avoidance of Single Points of Failure 28
2.7.3 Microservices 28
2.7.4 Cross-Platform Availability 28

3 User Interface Design **29**

4 Requirements Traceability **31**

5 Implementation, Integration and Test Plan **33**
5.1 Overview and Implementation Plan 33
5.2 Features identification 33
5.3 Integration Strategy 33
5.4 System Testing Strategy 33

6 Effort Spent **35**

7 References **37**

List of Figures **39**

List of Tables **41**

1 Introduction

1.1. Purpose

The purpose of this document is to serve as a detailed descriptor of the BBP project, to be used by actual developers and as an actual agreement between customer and contractors: it will contain detailed informations about the architecture of the system, its full functionalities, constraints and it will allow the customer to validate the integrity and completeness of said system. With this DD, the customer will also be able to validate all of the requirements, to verify if the system finally meets the expectations.

1.2. Scope

The Best Bike Paths (BBP) project, initiated by a leading bike association, aims to be an advanced support system for the cycling community designed to thrive within a transport landscape dominated by cars.

The core idea is to foster a collaborative digital environment where cyclists can actively record, consult, and publish both their own and other users' riding paths.

This platform will serve as the definitive centralized resource for finding highly recommended routes tailored precisely to the user's specific experience, preferences, and safety considerations.

Ultimately, the system aims to significantly enhance the quality of the cycling experience by providing verified, community-rated routes, thereby connecting riders and making high-quality path discovery universally accessible

1.3. Definitions, Acronyms, Abbreviations

- BBP: Best Bike Paths
- RASD: Requirements Analysis and Specification Document
- DD: Design Document
- IIT: Implementation, Integration and Test Plan

- WAF: Web Application Firewall

1.4. Reference Documents

- Specification document assignment
- IEEE standard documentation for DD
- Materials for the Software Engineering 2 course made available on WeBeep

1.5. Document Structure

The document is organized in seven different chapters that will cover every detail on the actual design of our project:

- **Chapter 1 - Introduction:** this chapter shows the aim of the doucment, what it is needed to know before to actually start the read of the whole text and restates the scope of the project.
- **Chapter 2 - Architectural Design:** this chapter shows all of the main components of the whole system, including architectural choices and patterns used.
- **Chapter 3 - User Interface Design:** this chapter shows and explains how the mockup pages we created integrate all of the implemented functionalities.
- **Chapter 4 - Requirements Traceability:** this chapter shows how we ensured that every requirement needed to the system to work is being satisfied with relative design choices.
- **Chapter 5 - Implementation, Integration and Test Plan:** this chapter shows how all of the components are implemented and their integration inside the system, also providing a test plan for all of their instances.
- **Chapter 6 - Effort Spent:** this chapter includes a table where the amount of hour of commitment of every participant is shown relative to its section.
- **Chapter 7 - References:** finally, the last chapter contains all of our document references used to better refine our DD.

2 | Architectural Design

2.1. Overview

Here we represent an overview of how the entire BBP architecture is composed of:

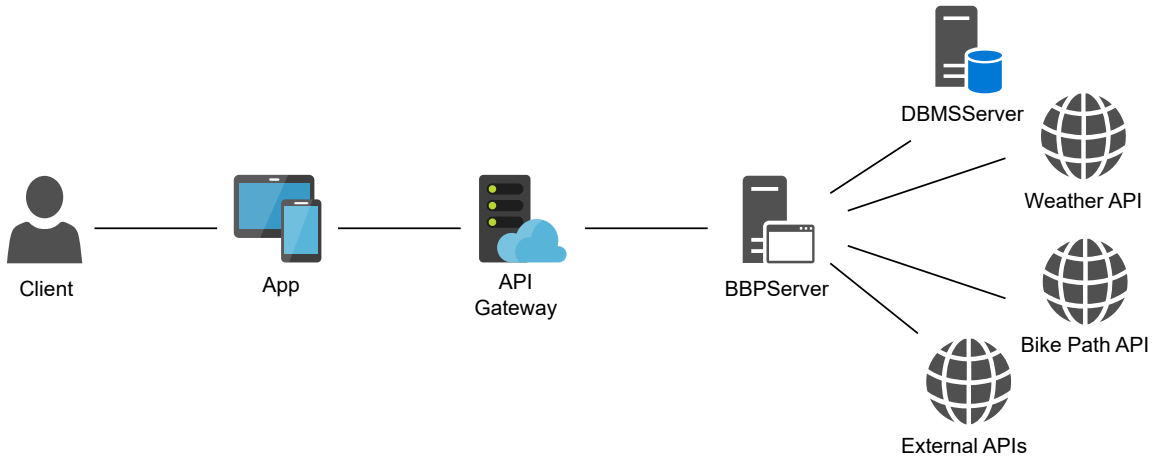


Figure 2.1: Overview

2.1.1. Client Side

- **Client**: Represents the end-user, categorized as either a Registered User or a Guest (Unregistered User). They interact with the platform to search for routes, record trips, or contribute road status data
- **App**: A mobile application, compatible with Android and iOS, that serves as the primary user interface. It is responsible for high-speed data acquisition from geospatial sensors (GPS) and Inertial Measurement Units (IMU) to track movement and detect road anomalies like potholes

2.1.2. Server Side

- **API Gateway**: Acts as the secure entry point for all requests originating from the mobile application. It manages communication using HTTPS/TLS protocols to ensure the

integrity of sensitive geolocation data transmitted to the backend. It is responsible for load balancing requests among BBP Server instance

- **BBP Server**: The central component of the system architecture that implements the core business logic. It handles Path Scoring and Ranking, resolves data conflicts during Information Merging, processes Automated Trip Recording guesses, and manages user authentication sessions
- **DBMS Server**: The Database Management System that serves as the persistent repository for the platform. It stores all data, including user profiles, saved routes, and system logs. It acts as the central repository for all essential information required by the platform
- **Weather API**: An interface with external meteorological services. It allows the system to retrieve real-time environmental data, like temperature and wind speed, to enrich trip logs with local weather context
- **Bike Path API**: A specialized external service utilized to Search Path. It provides the necessary data to visualize existing cycling networks
- **External APIs**: General category for third-party services. These are used for geocoding, street name resolution, and up-to-date base-map rendering to ensure the reliability of the road network presented to users, and most importantly to calculate the distance between points during recordings

2.2. Component View

2.2.1. High Level Diagram

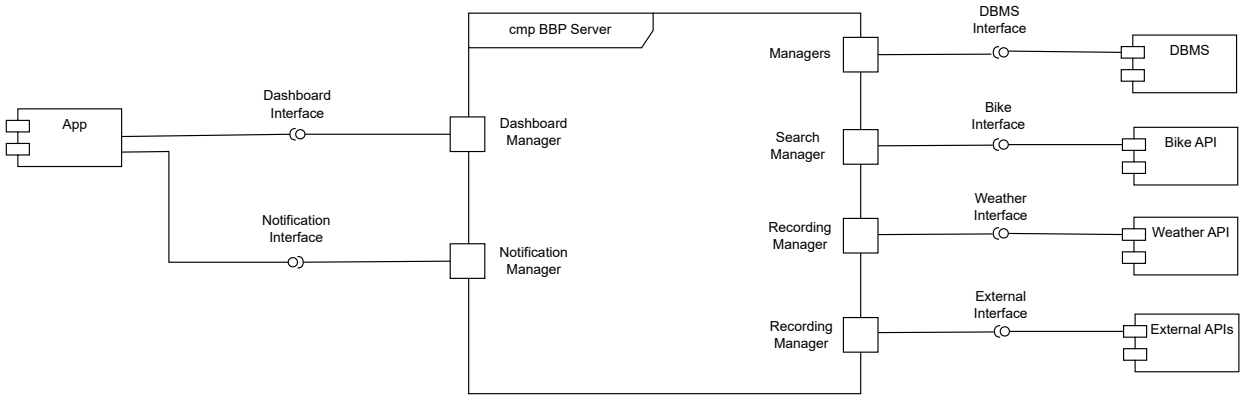


Figure 2.2: High Level Diagram

In the figure above is the high-level component diagram of BBP where it's represented the external components of BBP and how they communicate with the BBP server, in particular:

- **App**: This component represents the front-end interface used by cyclists to interact with the system. It communicates with the server via the Dashboard and Notification interfaces
- **DBMS (Database Management System)**: Connected via the DBMS Interface, this is the centralized repository for the system, it's used by different managers in the server application
- **Bike API**: The system communicates with this API to find cycling routes and retrieve up-to-date information regarding road networks and cycling-compatible paths, only used by search manager to respond request like search path in a zone
- **Weather API**: This dedicated service provides real-time meteorological conditions, such as temperature and wind speed, which the system uses to enrich trip data and help users analyze their performance relative to the environment. Only used by recording manager
- **External APIs**: These interfaces are used specifically for geocoding (converting coordinates to street names) and performing precise distance calculations between two GPS points. Only used by recording manager

2.2.2. Low Level Diagram

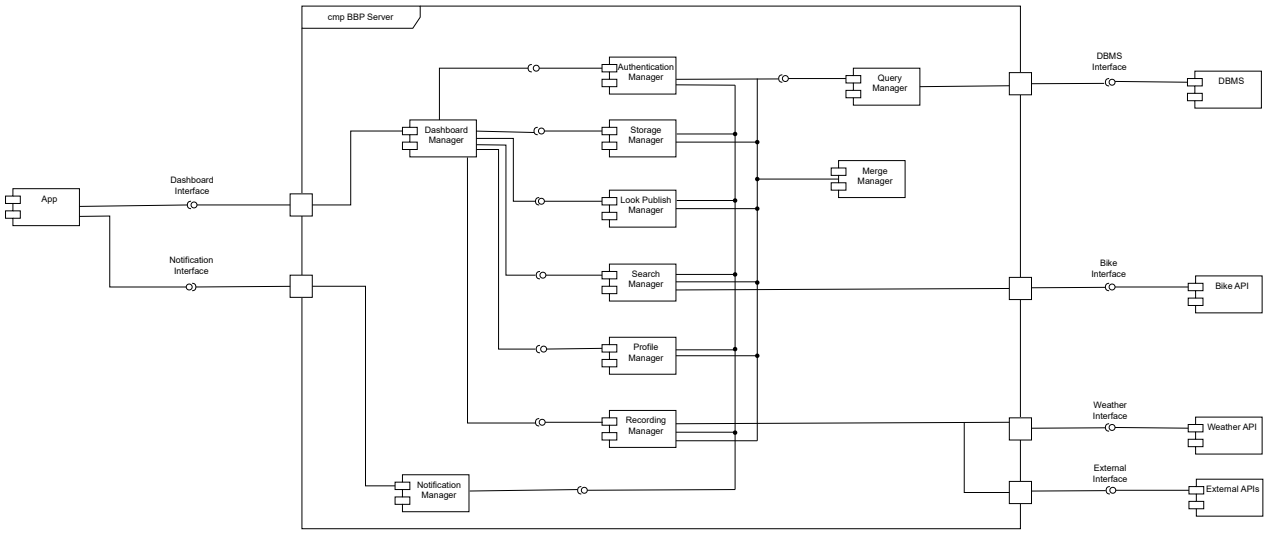


Figure 2.3: Low Level Diagram

The figure above represents the complete architecture of BBP with each components inside the BBP Server:

- **Authentication Manager**: Serves as the security gatekeeper for the BBP system. It is responsible for handling all procedures related to user identity verification, including the registration of new accounts and the secure login of existing users. This component validates credentials against stored data, manages session states (such as guest versus authenticated access), and ensures that only authorized users can access sensitive features like trip recording and history management
- **Dashboard Manager**: Acts as the central orchestration layer for the user interface, serving as the primary bridge between the mobile application (App) and the various backend sub-systems. It aggregates data from the storage, profile, and recording modules to present a cohesive view to the user, such as displaying the "Home" screen or summarizing recent activities. It ensures that the information requested by the Dashboard Interface is routed to the correct internal manager for processing
- **Storage Manager**: Is tasked with the persistence and retrieval of historical cycling data. It manages the user's personal log of completed trips, ensuring that GPS traces, timestamped statistics (speed, distance), and performance metrics are securely saved for future reference. This component allows users to access their "Trip History" and is responsible for differentiating between private data and data that has been flagged for public contribution
- **Look Publish Manager**: Facilitates the community-sharing aspects of the platform. It handles the logic required for browsing paths published by other users, allowing the system

to filter and retrieve "public" routes based on geographic areas. Furthermore, it manages the transition of a trip from a private recording to a public resource, ensuring that the user's intent to share their route with the community is processed and reflected in the system

- **Search Manager**: Engineers the route discovery process. It processes user requests for paths between specific origin and destination points, calculating optimal routes based on available data. This manager implements the scoring algorithms that evaluate paths not just by distance, but by the quality of the road surface and the absence of obstacles, identifying the "best" path as defined by the community data
- **Profile Manager**: Administers all user-specific data and preferences. It allows registered users to view and update their personal details, such as account credentials and privacy settings. Additionally, it aggregates cumulative statistics to provide the user with a comprehensive summary of their engagement with the platform
- **Recording Manager**: Orchestrates the active data acquisition lifecycle during a cycling session. It supports both "Manual Mode" (where users input data after a trip) and "Automated Mode" (which uses real-time GPS and sensor data). It interfaces with external APIs and device sensors to capture geospatial coordinates and detect anomalies like potholes in real-time, preparing this raw data for validation and eventual storage
- **Notification Manager**: Handles the dissemination of system alerts and prompts to the user. It is responsible for triggering push notifications, such as the alert asking a user to confirm if they wish to start recording when cycling speed is detected. It ensures that the user is prompted at the appropriate times to validate automated detections, such as confirming the presence of a pothole at the end of a trip
- **Merge Manager**: Implements the core logic for data consistency and conflict resolution. It is responsible for aggregating path information from multiple users regarding the same street segment. This manager applies algorithms to weigh new data against historical records—prioritizing data freshness and confirmation counts, to update the global "score" and status of a path, ensuring the community map reflects the most accurate current conditions
- **Query Manager**: Serves as the data access layer, abstracting the complexity of the database interactions from the other business logic components. It receives data requests from the various managers and translates them into operations compatible with the DBMS. It ensures that data is efficiently read from or written to the underlying database infrastructure while maintaining data integrity

Recording Manager

Here we describe the Recording Manager component in detail:

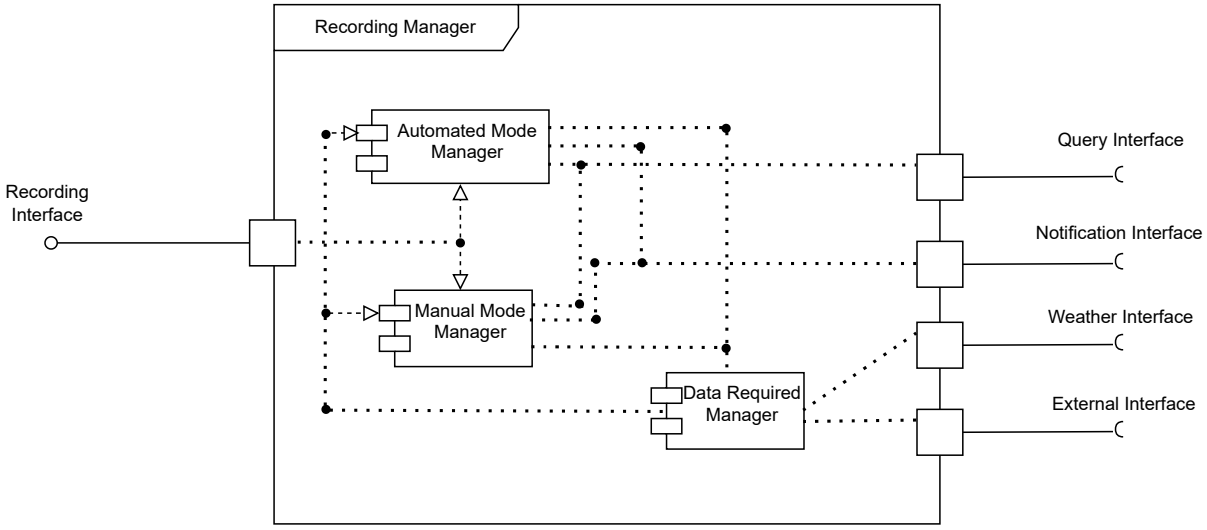


Figure 2.4: Recording Manager

- **Automated Mode Manager**: The Automated Mode Manager is responsible for the real-time acquisition of trip data without requiring constant user interaction. It implements the logic to infer cycling activity by monitoring the user's speed and initiates data collection when a specific threshold is exceeded. This component interfaces directly with the mobile device's sensors to capture GPS coordinates for path reconstruction, as well as accelerometer and gyroscope readings to detect potential path irregularities.
- **Manual Mode Manager**: The Manual Mode Manager handles the "offline reporting" workflow, allowing users to log cycling activities after they have occurred It provides the functionality for users to reconstruct their route by inputting a sequence of street names or selecting segments on a map, rather than relying on real-time GPS tracking. This component ensures that users can manually specify essential metadata, such as the total duration of the trip and the specific timeframes. Furthermore, it manages the annotation process, allowing users to manually insert status updates or report obstacles for specific segments of the reconstructed path
- **Data Required Manager**: The Data Required Manager acts as an enrichment engine that enhances basic trip data with context and precise metrics using external services. It interacts with the Weather Interface to retrieve historical meteorological conditions corresponding to the specific time and location of the recorded trip. Additionally, it utilizes the External Interface to communicate with third-party mapping APIs for geocoding services and to calculate precise distances between coordinates, ensuring that the statistics displayed to the user, such as total distance covered and elevation gain, are accurate

2.3. Deployment View

In this section of the document we focus on the deployment architecture of BBP and the motivations behind our design choices.

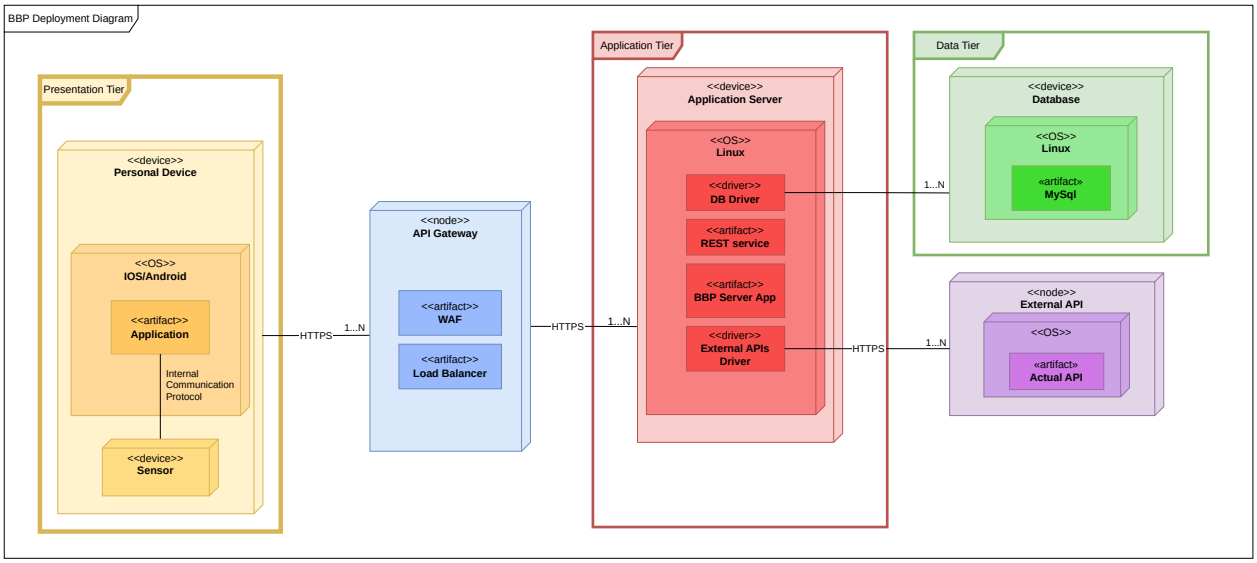


Figure 2.5: Deployment View

We divided our system into three main tiers—presentation, application, and data tiers (better explained in Section 2.6)—to better isolate common features between all tangible and intangible instances of the objects involved. Below is a list of all actors and components in the system:

- **Personal Device**: a personal device consists of a sensor device (here modeled as the union of all the device's movement sensors) and an operating system that can be either Android or iOS. It is intended as a personal piece of hardware such as a smartphone or smartwatch carried by the user throughout the execution of the application. The personal device communicates with the Application Server through a local instance of the application: every request made by the Application to the Application Server follows a RESTful interaction style over HTTPS.
- **API Gateway**: it is a node offered by a third party (such as AWS or Google Cloud Services) that provides a built-in Web Application Firewall (WAF) and a load balancer to distribute traffic among multiple Application Servers, thus avoiding single points of failure.
- **Application Server**: it is a device that contains the core logic of the application and connects all the drivers required for the application to operate. It communicates with external APIs using REST and interacts with multiple databases through a continuously managed connection pool.
- **Database**: we opted for multiple databases (one primary database and several read-

only replicas) with a connection pool managed by the Application Server, ensuring that databases are protected by a limited number of open connections. This approach minimizes concurrency issues and prevents overload. The number of connections can be adjusted according to traffic patterns to achieve an optimal balance between hardware usage and system efficiency. Linux was chosen as the operating system due to its open-source nature, and MySQL for its robustness.

A closer look at some of the internal components of the system follows:

- **Application Artifact**: the client-side application, primarily used as the presentation layer. It stores a signed JWT (JSON Web Token) used for authentication and is connected to all sensors required to detect movements in automated mode using cross-platform mobile frameworks.
- **BBP Server App**: the core model of the application. It contains the business logic and ensures that all operations are coherent with user requests by following the Model-View-Controller (MVC) pattern. It hosts all managers and connects them to the necessary drivers used to interface with external tools.

2.4. Runtime View

This section illustrates how the architectural components interact to fulfill the primary user scenarios. For each scenario, a description of the execution flow is provided, referencing the corresponding sequence of interactions.

2.4.1. User Login

This scenario describes how a registered User authenticates into the system to access private functionalities.

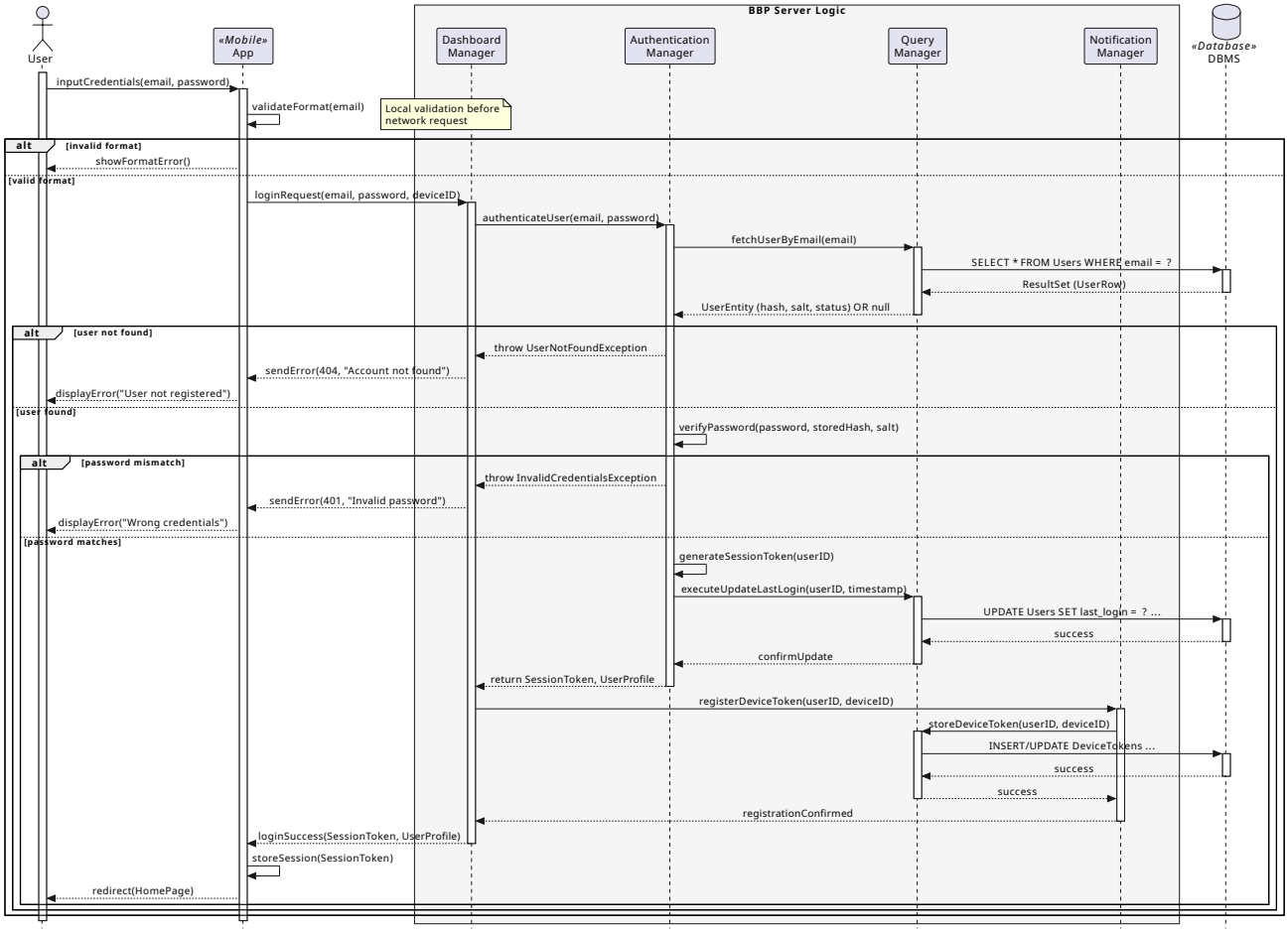


Figure 2.6: Runtime view for 'User Login'

Description of the flow: The process begins when the User inputs their credentials (email and password) into the `App`. The `App` performs a preliminary local validation (e.g., format checking) before initiating network communication. If valid, the `App` sends a request containing the credentials and the current `DeviceID` to the `Dashboard Manager`.

The `Dashboard Manager`, acting as the system entry point, delegates the identity verification to the `Authentication Manager`. To retrieve the user's stored data (such as password hash and salt), the `Authentication Manager` invokes the `Query Manager`, which executes the specific `SELECT` query against the `DBMS`.

Upon receiving the user data:

- **If the user is not found or the password hash does not match:** The `Authentication Manager` throws a specific exception. The `Dashboard Manager` catches this and returns an appropriate error message (e.g., 401 Unauthorized) to the `App`.
- **If the credentials are valid:**
  1. The `Authentication Manager` generates a new Session Token.

2. It invokes the `Query Manager` again to update the "Last Login" timestamp in the `DBMS`.

3. Control returns to the `Dashboard Manager`. Before finalizing the response, the `Dashboard Manager` coordinates with the `Notification Manager` to register the User's `DeviceID`, enabling future push notifications. The `Notification Manager` persists this association via the `Query Manager`.

4. Finally, the `Dashboard Manager` returns the Session Token and User Profile to the `App`, which stores the token and redirects the user to the homepage.

2.4.2. User Registration

This scenario details the procedure for onboarding a new User into the system. It involves a coordinated interaction between authentication mechanisms, profile management, and notification services to ensure a complete account setup.

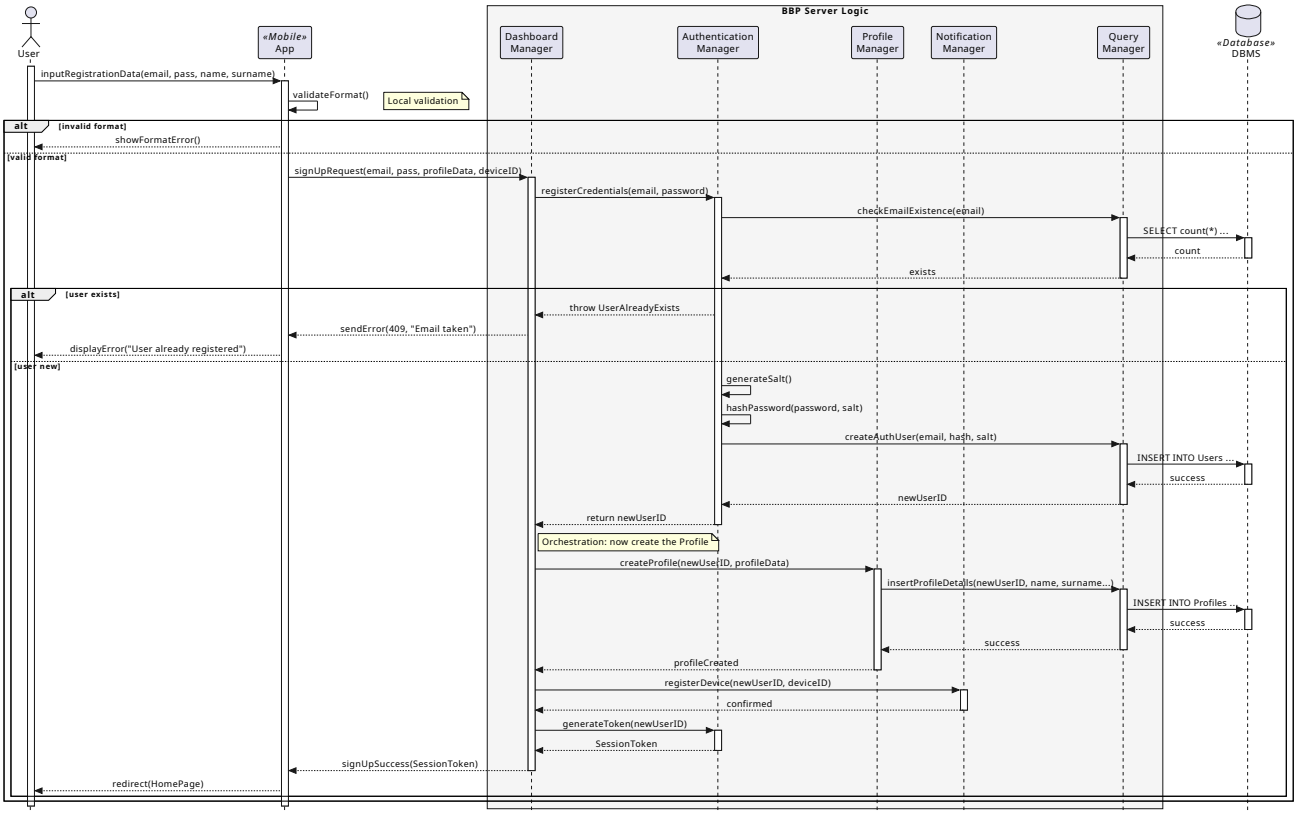


Figure 2.7: Runtime view for 'User Registration'

Description of the flow: The process begins when the User submits their registration data (email, password, and personal details such as name and surname) via the `App`. The `App` executes a local validation logic (e.g., password strength, email format) to filter out malformed requests.

If the data is valid, the App sends a composite request to the Dashboard Manager, including the current DeviceID.

The Dashboard Manager acts as the central orchestrator for this multi-step transaction, delegating specific responsibilities to the appropriate sub-components:

- **Step 1: Credential Verification and Creation.** The Dashboard Manager first invokes the Authentication Manager. This component uses the Query Manager to check if the provided email is already registered in the DBMS.
  - If the user exists, an exception is thrown, and the Dashboard Manager returns a conflict error (e.g., 409 Conflict) to the App.
  - If the user is new, the Authentication Manager hashes the password and instructs the Query Manager to persist the new security credentials. It then returns the new unique UserID to the Dashboard Manager.
- **Step 2: Profile Initialization.** Upon successful credential creation, the Dashboard Manager calls the Profile Manager. This component takes the personal information (name, surname, etc.) and the newly created UserID to initialize the user's business profile in the DBMS via the Query Manager. This ensures a strict separation between authentication data and user profile data.
- **Step 3: Device Registration and Auto-Login.** To complete the onboarding:
  1. The Dashboard Manager interacts with the Notification Manager to register the User's DeviceID, enabling immediate push notification capabilities.
  2. It requests a new Session Token from the Authentication Manager to allow for an "auto-login" experience, so the user does not need to re-enter credentials immediately after signing up.
  3. Finally, the Dashboard Manager aggregates the result and sends the Session Token back to the App, which redirects the User to the homepage.

2.4.3. User Logout

This scenario illustrates the secure termination of a User's session. It is designed as a symmetric operation to the login process, ensuring that both the security context (session validity) and the operational context (notification delivery) are properly dismantled.

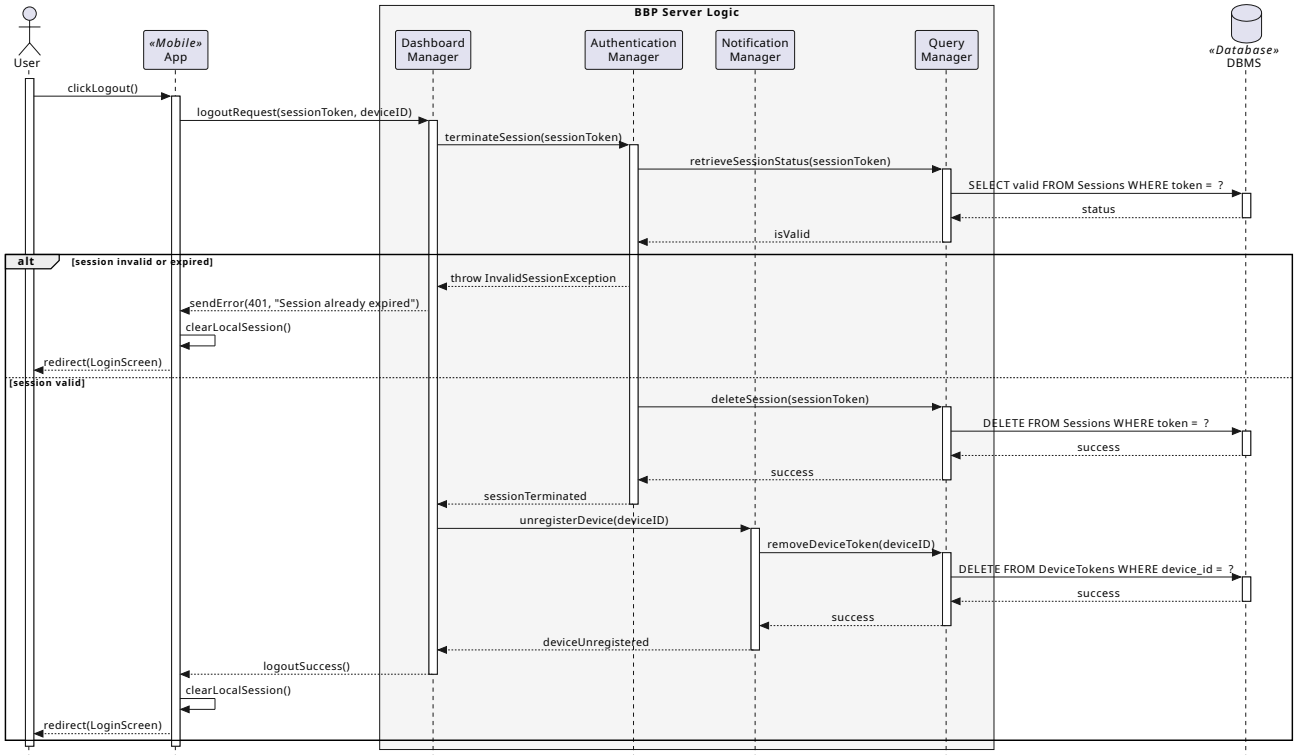


Figure 2.8: Runtime view for 'User Logout'

Description of the flow: The process is initiated when the User selects the logout option within the `App`. The `App` constructs a termination request containing the current Session Token and the Device ID, transmitting it to the `Dashboard Manager`.

The `Dashboard Manager` orchestrates the teardown of the user's active context through two main phases:

- **Phase 1: Session Invalidation.** The `Dashboard Manager` first instructs the `Authentication Manager` to terminate the session.
  - The `Authentication Manager` queries the `DBMS` via the `Query Manager` to verify the token's validity.
  - If valid, the token is physically removed or marked as expired in the database. This step is critical to prevent session replay attacks. If the session is already invalid (e.g., expired), the system handles the exception gracefully, ensuring the client is still forced to a logged-out state.
- **Phase 2: Notification Deregistration.** To ensure no further personal alerts are sent to the device, the `Dashboard Manager` coordinates with the `Notification Manager`.
  - The `Notification Manager` invokes the `Query Manager` to remove the association between the user and the specific `DeviceID` in the `DBMS`. This mirrors the registration

step performed during login/signup.

Upon the successful completion of server-side cleanup, the `Dashboard Manager` sends a confirmation to the `App`. The `App` then clears all local storage (cached tokens, user data) and redirects the User to the login screen.

2.4.4. Manual Trip Recording

This scenario describes the process by which a Registered User manually reconstructs a trip by inputting a sequence of nodes and metadata.

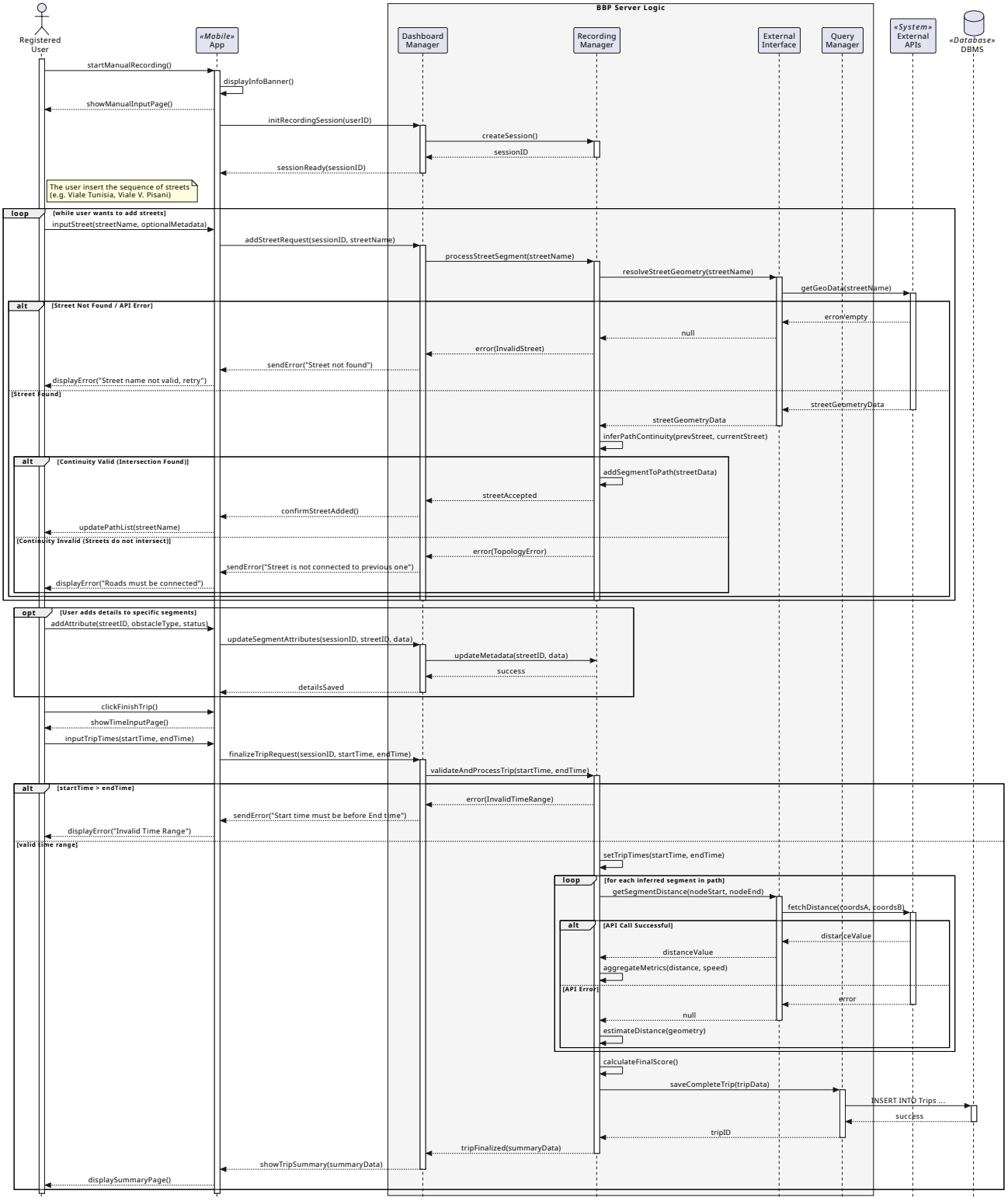


Figure 2.9: Runtime view for 'Manual Trip Recording'

Description of the flow: The interaction begins when the User selects the "Manual Mode" on the `App`. After acknowledging the introductory information, the process unfolds in three distinct phases managed by the `Recording Manager` via the `Dashboard Manager`:

- **Phase 1: Path Reconstruction by Streets.** The User inputs the sequence of **street names** traversed during the trip.
  - For each inserted street, the `Recording Manager` interacts with the `External Interface` to validate the street's existence and retrieve its geometry.
  - Crucially, the system performs a **topology check** (inference) to ensure the new street physically intersects or connects with the previous one.
  - Upon successful validation of continuity, the street is added to the path list. The User can optionally associate metadata (e.g., obstacles or road status) to specific street segments. If a street is not found or not connected, the system prompts the User for correction.
- **Phase 2: Temporal Definition.** Once the sequence of streets is defined, the User concludes the session and manually inputs the **start and end times** of the trip.
  - The `Recording Manager` validates that the start time strictly precedes the end time. Invalid ranges trigger an error response, prompting the User to re-enter the temporal data.
- **Phase 3: Data Enrichment and Persistence.** Before saving, the system automatically enhances the reconstructed path:
  1. The `Recording Manager` iterates through the inferred segments (derived from street intersections) and invokes the `External Interface` to request accurate distance metrics from the `External APIs`.
  2. If the external service fails (e.g., timeout), the system attempts to estimate the distance based on the retrieved geometry to ensure process resilience.
  3. Once all metrics are aggregated (calculating total distance and average speed based on the time range), the `Recording Manager` uses the `Query Manager` to persist the complete Trip object into the `DBMS`.
  4. Finally, a summary of the recorded activity is returned to the `App` for display.

2.4.5. Automated Trip Recording

This scenario details the "hands-free" recording mode, where the system leverages device sensors to detect activity and automates data collection. It features a complex flow involving local hardware interaction and concurrent server-side processes to enrich trip data in real-time.

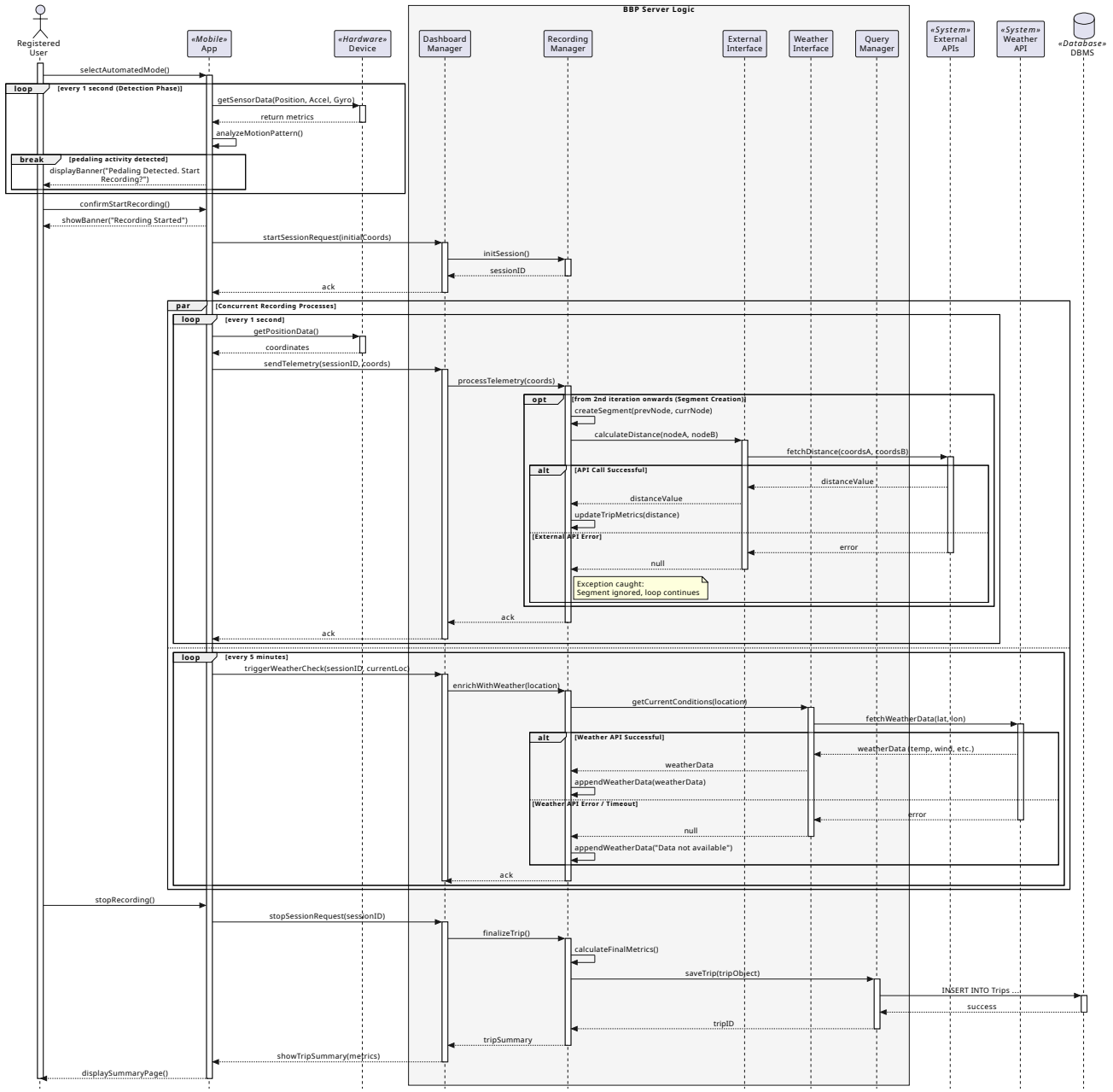


Figure 2.10: Runtime view for 'Automated Trip Recording'

Description of the flow: The workflow is initiated when the User activates "Automated Mode" on the `App`. The process is structured into three logical phases, distinguishing between local detection and server-side processing:

- **Phase 1: Motion Detection and Initialization.**
  - The `App` enters a monitoring loop, periodically querying the local `Device` for sensor metrics (accelerometer, gyroscope, and position).
  - Upon detecting a pattern consistent with cycling, the `App` prompts the User for confirmation.

  - Once confirmed, the `App` sends an initialization request to the `Dashboard Manager`, which instructs the `Recording Manager` to create a new active session.

- **Phase 2: Concurrent Recording Loops.** During the active trip, the system orchestrates two parallel execution threads to optimize data accuracy and API usage:

  1. **High-Frequency Telemetry Loop (every 1s):** The `App` streams GPS coordinates to the `Recording Manager`. For every new segment detected, the system invokes the `External Interface` to fetch precise distance metrics from `External APIs`. Exception handling is in place to skip segments if the external service is unreachable, ensuring continuous recording.

  2. **Low-Frequency Environmental Loop (every 5 mins):** To enrich the trip data without exhausting API quotas, the `Recording Manager` periodically triggers a weather check. It uses the `Weather Interface` to retrieve conditions from the `Weather API`. If the API fails, the system gracefully defaults to a "Data not available" state rather than interrupting the flow.

- **Phase 3: Termination and Persistence.**

  - When the User manually stops the recording, the `App` signals the termination to the `Dashboard Manager`.

  - The `Recording Manager` finalizes the calculations (aggregating distances and speeds) and invokes the `Query Manager` to persist the complete Trip object into the `DBMS`.

  - Finally, a summary of the performance metrics is returned to the User.

2.4.6. Personal Trip Storage

This scenario illustrates the management of the User's personal history of trips. It covers the retrieval of stored data and, crucially, the mechanism for toggling the visibility of a trip between "Private" (visible only to the owner) and "Public" (searchable by the community).

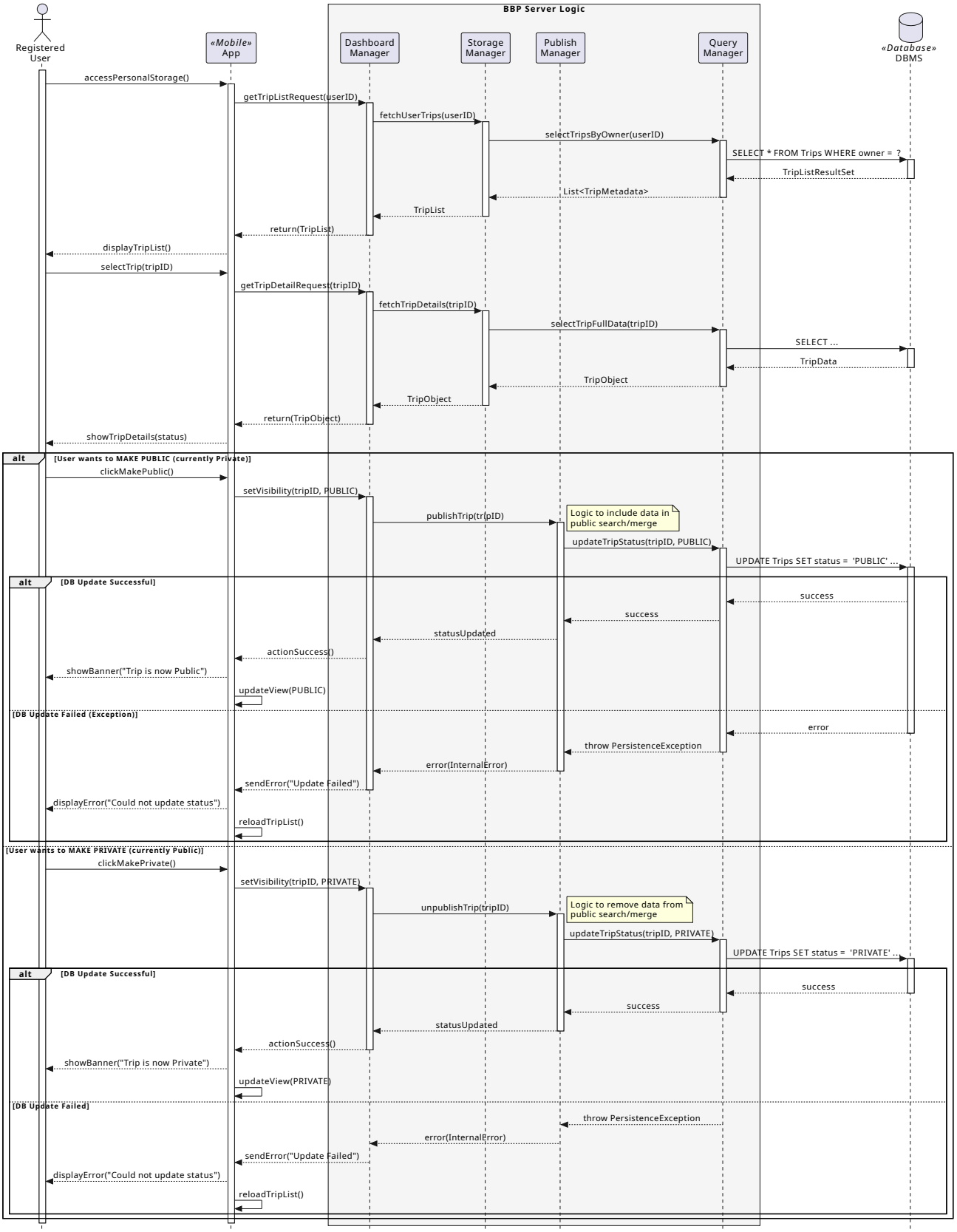


Figure 2.11: Runtime view for 'Personal Trip Storage'

Description of the flow: The interaction is structured into two main logical phases: data

retrieval and status modification.

- **Phase 1: Retrieval of Trip Data.** When the User accesses the storage section, the `Dashboard Manager` delegates the request to the `Storage Manager`. This component interacts with the `Query Manager` to fetch the metadata list of trips owned by the specific User from the `DBMS`. A similar flow applies when the User requests the full details of a specific trip.
- **Phase 2: Visibility Management (Publication/Unpublication).** The core complexity lies in changing a trip's status. When the User toggles the visibility (e.g., "Make Public" or "Make Private"):
  1. The request is routed to the `Publish Manager` (distinct from the Storage Manager). This component validates the request and invokes the `Query Manager` to execute an atomic `UPDATE` on the trip's status field in the `DBMS`.
  2. **Success Path:** If the database confirms the update, the system acknowledges the change. For a "Make Public" action, this logically implies the trip becomes available for the system's "Data Merge" algorithms. The `App` updates the view immediately.
  3. **Failure Path:** As shown in the diagram's alternative flows, if a persistence error occurs (e.g., database lock or connection failure), the system prioritizes consistency. The `Publish Manager` propagates the exception, the `Dashboard Manager` returns an error, and the `App` forces a reload of the list. This ensures the User never sees a "false positive" state where the UI shows "Public" while the DB remains "Private".

2.4.7. Path Search

This scenario illustrates the workflow for retrieving bike paths based on geographical constraints. It emphasizes the validation logic performed by the backend to ensure data consistency before executing complex search queries.

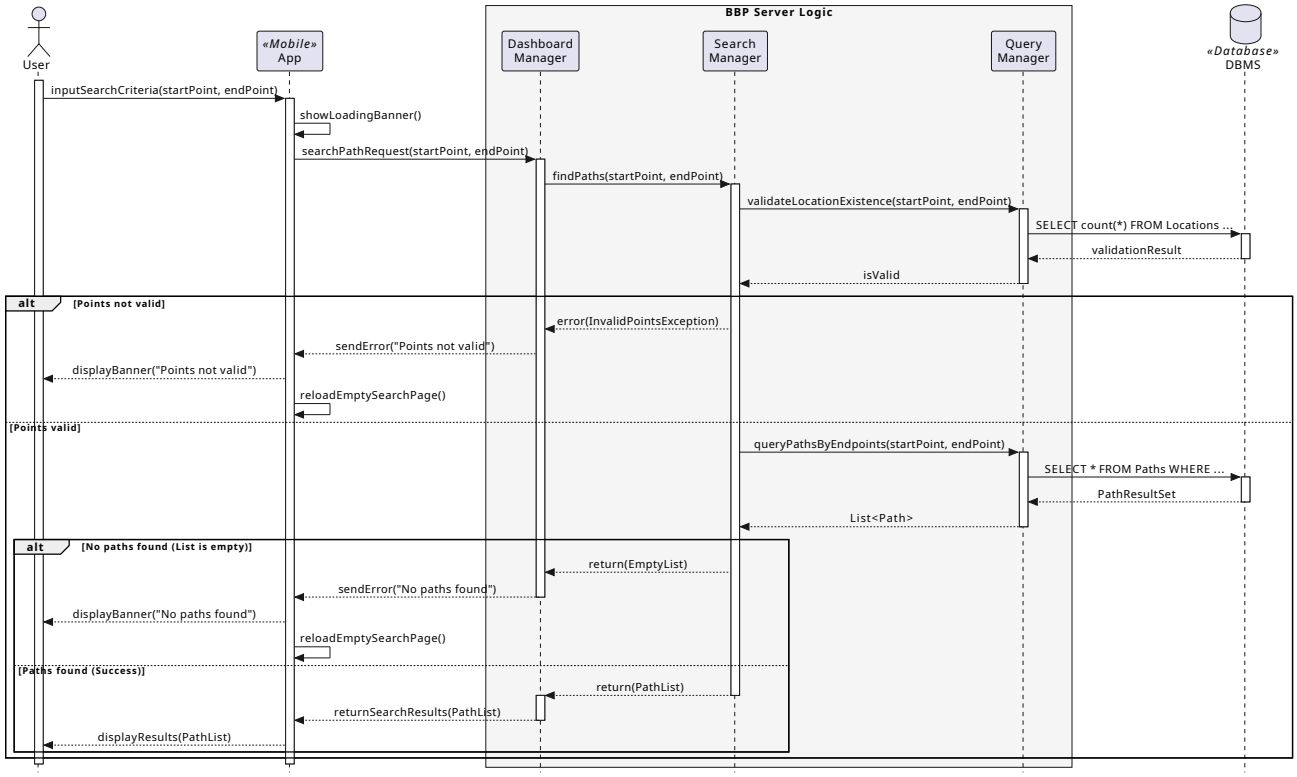


Figure 2.12: Runtime view for 'Path Search'

Description of the flow: The interaction begins when the User inputs the desired start and end points via the `App` interface. The `App` displays a loading state and forwards the search criteria to the `Dashboard Manager`.

The `Dashboard Manager` routes the request to the `Search Manager`, which orchestrates the retrieval process in two distinct logical steps to optimize resource usage:

- **Step 1: Input Validation.** Before searching for paths, the `Search Manager` invokes the `Query Manager` to verify that both the start and end points correspond to valid locations existing in the `DBMS`.
  - If one or both points are invalid, the `Search Manager` raises an exception. The `App` receives an error message (e.g., "Points not valid"), displays a specific banner to the User, and reloads the search page to allow for correction.
- **Step 2: Path Retrieval.** If the points are valid, the `Search Manager` instructs the `Query Manager` to execute the specific query to find all paths connecting the two locations.
  - **No Paths Found:** If the query returns an empty result set, the system notifies the `App`. The User is informed via a "No paths found" banner, and the interface is reset for a new search.
  - **Search Successful:** If matching paths are found, the `Query Manager` returns the

list of path objects. The `Search Manager` forwards this data through the `Dashboard Manager` to the `App`, which renders the results list for the User to browse.

2.4.8. Browsing of Published Paths

This scenario depicts the workflow allowing Users to explore the catalog of publicly available bike paths within a specific geographical area. It highlights the filtering logic that ensures only content explicitly marked as "Public" is accessible to the general user base.

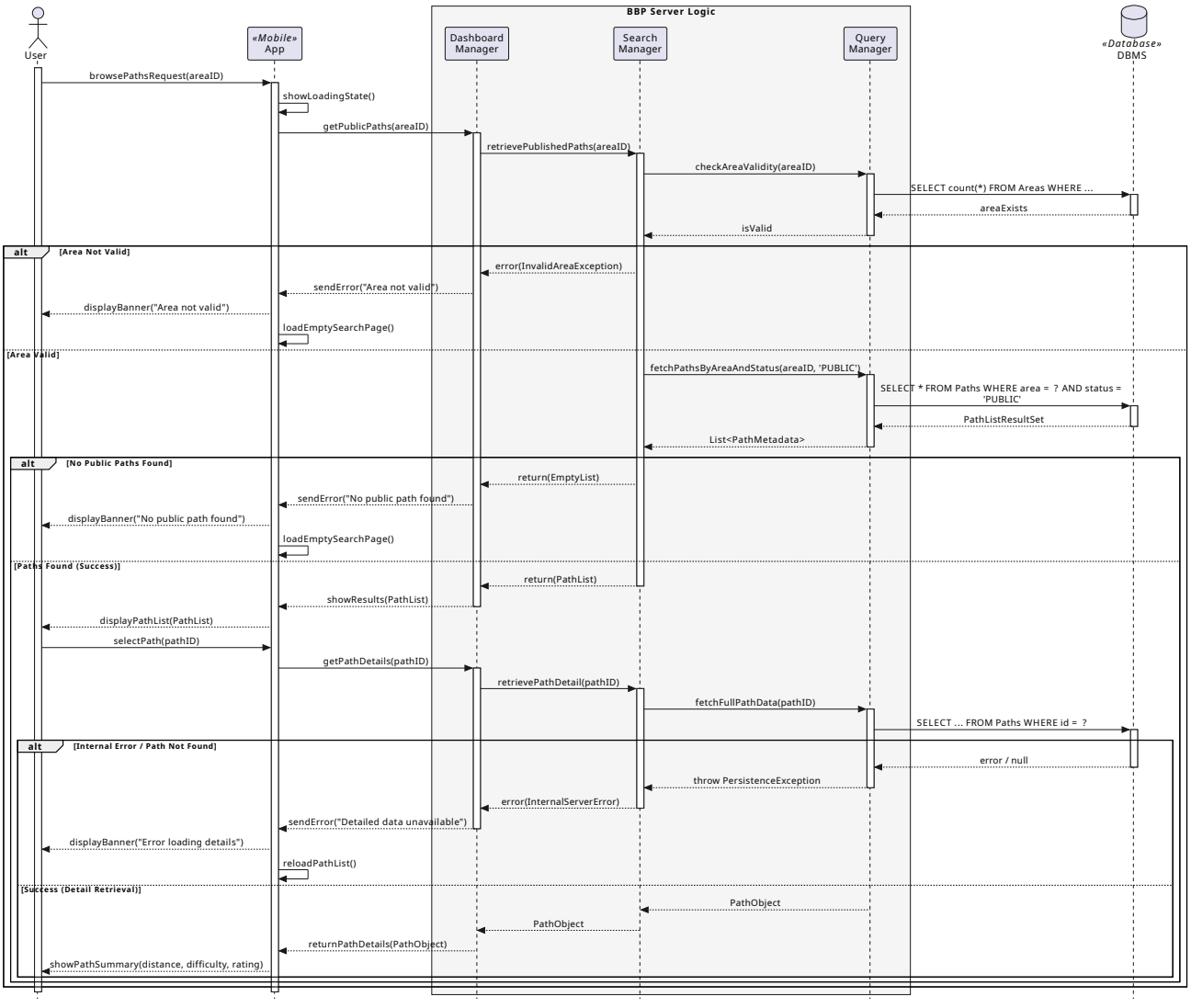


Figure 2.13: Runtime view for 'Browsing of Published Paths'

Description of the flow: The interaction begins when the User selects a geographical area of interest via the `App`. The interface displays a loading state while the request is forwarded to the `Dashboard Manager`, which delegates the operation to the `Search Manager`.

The retrieval process is orchestrated in three sequential phases to ensure data consistency and

validity:

- **Phase 1: Area Validation.** Before attempting any data retrieval, the `Search Manager` invokes the `Query Manager` to verify the existence of the requested area in the `DBMS`.
  - If the area ID is malformed or non-existent, the process terminates immediately, and the `App` prompts the User to select a valid location.
- **Phase 2: Public Paths Retrieval.** Upon successful validation, the `Search Manager` requests the list of paths associated with that area.
  - Crucially, the query executed by the `Query Manager` includes a strict filter for `status = 'PUBLIC'`. Private trips stored by other users are strictly excluded at the database level.
  - If the query returns results, the list is displayed to the User. If the area contains no public paths, a specific notification is returned.
- **Phase 3: Detail View.** The workflow concludes with the User selecting a specific path from the list.
  1. The `App` requests the full details (including metrics like difficulty and rating) for the selected ID.
  2. The `Search Manager` retrieves the complete object via the `Query Manager`.
  3. Should an internal consistency error occur (e.g., the path was deleted moments before selection), the system handles the exception by notifying the User and refreshing the list, rather than crashing the application.

2.4.9. Obstacles Detection

This scenario details the advanced telemetry analysis performed during an automated recording session. It illustrates how the system leverages hardware sensor data (accelerometer and gyroscope) to identify potential road hazards and includes a human-in-the-loop verification step to ensure data accuracy.

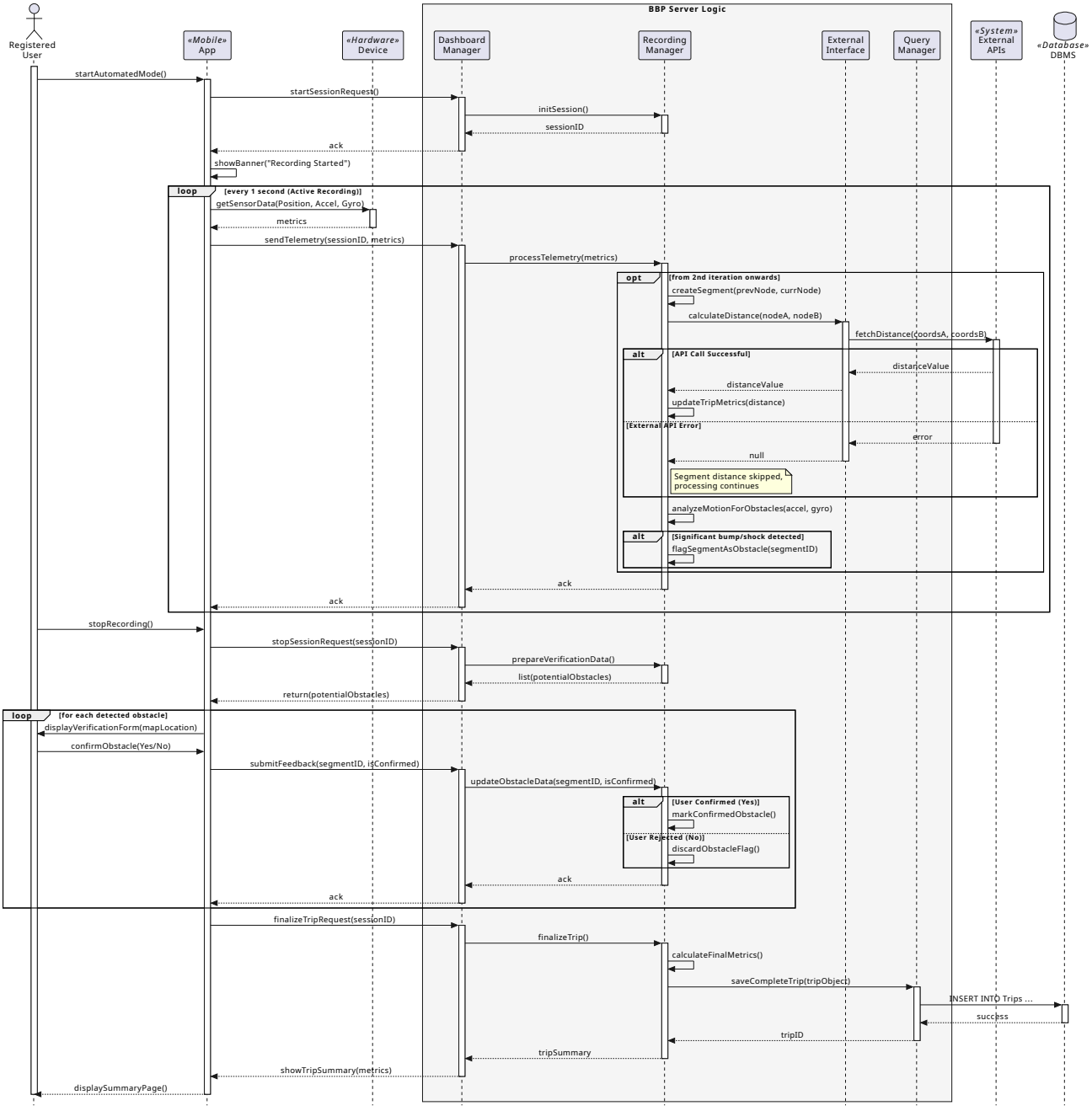


Figure 2.14: Runtime view for 'Obstacles Detection'

Description of the flow: The workflow is an extension of the Automated Recording mode. Once the recording session is initialized via the `Dashboard Manager`, the process executes in two distinct temporal stages:

- **Stage 1: Real-Time Detection and Enrichment.** During the active trip, the `App` streams sensor metrics to the `Recording Manager` at one-second intervals.

    1. **Distance Calculation:** The system invokes the `External Interface` to fetch segment lengths from `External APIs`. As depicted in the diagram, this interaction is

designed to be resilient: if the external service returns an error, the specific segment is skipped, allowing the recording loop to continue uninterrupted.

2. **Anomaly Recognition:** Simultaneously, the `Recording Manager` analyzes the accelerometer and gyroscope data. If a motion pattern consistent with a road anomaly (e.g., a pothole or bump) is detected, the current segment is internally flagged as a "Potential Obstacle".

- **Stage 2: Post-Trip User Verification.** When the User stops the recording, the system does not immediately save the trip but initiates a verification sequence.
  1. The `Recording Manager` returns the list of flagged segments to the `App`.
  2. The `App` presents these locations to the User, asking for confirmation. The User iterates through the list, confirming ("Yes") or rejecting ("No") the presence of an obstacle.
  3. The `Dashboard Manager` collects this feedback and updates the segment data accordingly, filtering out false positives.
- **Stage 3: Persistence.** Once the verification loop is complete, the `Recording Manager` calculates the final trip metrics and invokes the `Query Manager` to persist the validated Trip object (including confirmed obstacles) into the `DBMS`.

2.5. Component Interfaces

2.6. Styles and Patterns

BBP will be developed using the client-server paradigm integrated with a three-tier architecture and the Model-View-Controller pattern. The goal of this combined architecture is to achieve a clear separation of concerns among system components and improve control over each tier. The tiers involved are:

- **Presentation Tier**: this tier serves as the view component of the application and contains software that interacts directly with the user's hardware (i.e., the personal device and its sensors). It represents the top-level tier and is designed to be mostly stateless, although some user information is stored through JWT tokens for authentication. Its main functionality is to display information and available actions to the user, as well as present outputs generated by the backend. The presentation tier does not contain core business logic, preventing users from modifying it locally to perform unauthorized actions and keeping the application lightweight in terms of memory usage. To ensure platform independence, Flutter was adopted to provide a unified abstraction layer for accessing sensor information

across operating systems.

- **Application Tier**: this tier contains the core logic of the system and is composed of the Application Servers. It validates and executes user actions according to the current system state. Acting as a middle tier, it processes data received from the presentation tier and returns meaningful information to the user. Multiple Application Servers are deployed to enable scalability; with the use of an API Gateway equipped with a load balancer, requests can be routed consistently to specific servers using the JWT as an identifier. In case of server failure, users can be transparently rerouted to other servers, avoiding downtime.
- **Data Tier**: this tier represents the main repository of user and application data. It consists of a primary database and multiple secondary read-only replicas, all accessed through a connection pool that regulates the number of active connections. This architecture avoids data-tier downtime by allowing replica promotion in case of primary database failure, while preventing data inconsistency issues that may arise from having multiple writable databases.

In addition to the three tiers, a third-party API Gateway connects the presentation tier to the application tier. To satisfy scalability and security requirements, the API Gateway includes a **load balancer** to distribute incoming connections and a **Web Application Firewall** to protect against common attacks. AWS and Google Cloud Services are examples of providers offering such solutions.

The overall architecture follows a **client-server** interaction model in which most operations are event-driven. The client initiates the majority of transactions, except for server-generated notifications (e.g., automated mode activation). All client requests are validated by server-side logic, ensuring that communication errors and malicious behaviors are intercepted. Since communication is handled through **REST** over HTTPS, error messages can be clearly encoded and traced in both front-end and back-end scenarios.

The primary design pattern adopted is **Model-View-Controller**. The Model contains the logical components that manage and process application data, the View represents the graphical interface presented to the user through the client application, and the Controller handles user input and coordinates interactions between the Model and the View.

2.7. Other Design Decisions

2.7.1. JWT

JWT was chosen for authentication because, when combined with the load balancer deployed within the API Gateway, it enables secure login and registration while supporting scalability. Token expiration times (time-to-live) can be customized to allow either long-lived sessions or

frequent re-authentication, providing flexibility in security policies.

2.7.2. Avoidance of Single Points of Failure

The use of an API Gateway, a homogeneous set of Application Servers, and load-balanced routing eliminates single points of failure. This design allows the system to scale horizontally by adding servers, thereby increasing robustness and the ability to handle higher request volumes.

2.7.3. Microservices

A microservices-based architecture was adopted to improve functional scalability. Each manager is implemented as an isolated service coordinated through notification and query mechanisms. This approach facilitates targeted testing, simplifies maintenance, and improves error identification and resolution.

2.7.4. Cross-Platform Availability

By using Flutter, a single codebase is maintained for the client-side application, improving code readability and maintainability through a shared operating system abstraction. This choice also enables BBP to reach a wider user base by supporting multiple platforms.

3 | User Interface Design

4 | Requirements Traceability

5 | Implementation, Integration and Test Plan

5.1. Overview and Implementation Plan

5.2. Features identification

5.3. Integration Strategy

5.4. System Testing Strategy

6 Effort Spent

A table representing the total hours of work is shown here.

Work Areas	Alessandro Burato	Gabriele Crimella	Davide Faccini
Introduction	1h	0h	0h
Architectural Design	8h	0h	0h
UI Design	0h	0h	0h
Requirements Traceability	0h	0h	0h
IIT	0h	0h	0h
Group Meetings and Reviews	0h	0h	0h

Table 6.1: Table of Effort

7 | References

List of Figures

2.1 Overview . . . 3
2.2 High Level Diagram . . . 5
2.3 Low Level Diagram . . . 6
2.4 Recording Manager . . . 8
2.5 Deployment View . . . 9
2.6 Runtime view for 'User Login' . . . 11
2.7 Runtime view for 'User Registration' . . . 12
2.8 Runtime view for 'User Logout' . . . 14
2.9 Runtime view for 'Manual Trip Recording' . . . 16
2.10 Runtime view for 'Automated Trip Recording' . . . 18
2.11 Runtime view for 'Personal Trip Storage' . . . 20
2.12 Runtime view for 'Path Search' . . . 22
2.13 Runtime view for 'Browsing of Published Paths' . . . 23
2.14 Runtime view for 'Obstacles Detection' . . . 25

List of Tables

6.1 Table of Effort 35